Regime-Master (HMMBOT) — Master Architecture & Documentation

Version: Production (March 2026) **GitHub:** https://github.com/nikhildha/Synapticbots **Deployment:** Railway (Docker, PostgreSQL) **Exchanges:** CoinDCX (live), Binance Futures (paper/testnet)

Table of Contents

1. System Overview
2. High-Level Architecture
3. Python Trading Engine
   - 3.1 Main Bot Loop (main.py)
   - 3.2 Data Pipeline (data_pipeline.py)
   - 3.3 Feature Engine (feature_engine.py)
   - 3.4 HMM Brain (hmm_brain.py)
   - 3.5 Coin Scanner (coin_scanner.py)
   - 3.6 Risk Manager (risk_manager.py)
   - 3.7 Execution Engine (execution_engine.py)
   - 3.8 Order Flow Engine (orderflow_engine.py)
   - 3.9 Sentiment Engine
   - 3.10 Tradebook (tradebook.py)
   - 3.11 Engine API (engine_api.py)
4. Brain Logic — Deep Dive
5. Risk Management System
6. SaaS Dashboard (sentinel-saas)
   - 6.1 Technology Stack
   - 6.2 Application Pages
   - 6.3 API Routes
   - 6.4 Database Schema
   - 6.5 Authentication & Authorization
   - 6.6 Subscription System
   - 6.7 Exchange API Key Management
   - 6.8 Bot Session Lifecycle
7. Data Flow & Integration
8. Backtesting & Experimentation
9. Deployment & Infrastructure
10. Testing
11. Configuration Reference
12. Glossary

---

1. System Overview

Regime-Master is a production-grade algorithmic cryptocurrency trading system built on a 3-state Gaussian Hidden Markov Model (HMM) for market regime detection, combined with multi-factor conviction scoring, dynamic leverage management, and a full SaaS dashboard for multi-user deployment.

What It Does

- Scans up to 25 crypto coins every 5 minutes using HMM regime classification
- Scores each opportunity across 8 conviction factors (0–100 scale)
- Dynamically assigns leverage (0x to 35x) based on conviction score
- Opens long or short futures positions with multi-target partial profit booking (T1/T2/T3)
- Trails stop-losses using ATR-based dynamic rules
- Shuts down automatically if portfolio drawdown exceeds 10% in 24 hours
- Sends real-time Telegram alerts for all trade events
- Exposes full status and control via a Next.js SaaS dashboard with per-user bots

Key Design Principles

Principle	Implementation
Regime-aware trading	Only trades in BULL/BEAR; avoids CHOP for trend strategies
Evidence-gated leverage	Leverage unlocked only when conviction score ≥ 40
Anti-liquidation	Isolated margin, ATR stops, kill switch at 10% drawdown
Multi-target exit	25% booked at T1, 50% at T2, remainder runs to T3
Per-user isolation	Each SaaS user has their own bot, trades, API keys, and session history
Encryption at rest	Exchange API keys encrypted with AES-256-GCM before DB storage

---

2. High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                        RAILWAY CONTAINER                             │
│                                                                      │
│  ┌──────────────────────────┐    ┌──────────────────────────────┐   │
│  │   Python Trading Engine  │    │   Next.js SaaS Dashboard     │   │
│  │                          │    │   (Port 3000)                │   │
│  │  main.py (bot loop)      │◄───┤   React + TypeScript         │   │
│  │  hmm_brain.py            │    │   Prisma ORM → PostgreSQL    │   │
│  │  feature_engine.py       │    │   NextAuth (JWT sessions)    │   │
│  │  risk_manager.py         │    │   AES-256-GCM key storage    │   │
│  │  execution_engine.py     │    └──────────────┬───────────────┘   │
│  │  sentiment_engine.py     │                   │                   │
│  │  orderflow_engine.py     │    ┌──────────────▼───────────────┐   │
│  │  coin_scanner.py         │    │   Flask Engine API           │   │
│  │  tradebook.py            │◄───┤   (Port 5000)                │   │
│  │                          │    │   /api/bot-state             │   │
│  └──────────┬───────────────┘    │   /api/trades                │   │
│             │                    │   /api/close-trade           │   │
│             │                    │   /api/health                │   │
│  ┌──────────▼───────────────┐    └──────────────────────────────┘   │
│  │   /app/data/ (shared)    │                                       │
│  │   bot_state.json         │    ┌──────────────────────────────┐   │
│  │   tradebook.json         │    │   PostgreSQL Database        │   │
│  │   sentiment_log.csv      │    │   (Railway Postgres)         │   │
│  │   coin_tiers.csv         │    │   Users, Bots, Trades,       │   │
```

```
│  │   commands.json          │    │   Sessions, API Keys          │   │
│  └──────────────────────────┘    └───────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘
         │                                  │
         ▼                                  ▼
  ┌────────────────┐                  ┌────────────────┐
  │  CoinDCX API   │                  │   Binance API  │
  │  (Live Trade)  │                  │  (Paper/Test)  │
  └────────────────┘                  └────────────────┘
```

Component Responsibilities

Component	Language	Port	Responsibility
main.py	Python	—	Bot orchestration loop, cycle management
engine_api.py	Python/Flask	5000	REST API for dashboard ↔ engine communication
Next.js App	TypeScript	3000	SaaS dashboard, user management, API layer
PostgreSQL	SQL	5432	Persistent storage (users, bots, trades, sessions)
CoinDCX	External	—	Live futures trading (Indian exchange)
Binance Futures	External	—	Paper/testnet trading

---

3. Python Trading Engine

3.1 Main Bot Loop

File: `main.py` (56.5 KB)

The `RegimeMasterBot` class contains the entire bot lifecycle.

Timing Architecture

```
Every 30 seconds (LOOP_INTERVAL_SECONDS):
  └── Heartbeat loop
        ├── Read commands.json (START / STOP / PAUSE / CLOSE_ALL)
        ├── Update engine_state.json (health, uptime, CPU)
        └── Every 300 seconds (ANALYSIS_INTERVAL_SECONDS = 5 min):
              └── Full analysis cycle
                    ├── Coin scanner: refresh top coins (every 4 cycles = 20 min)
                    ├── For each coin:
                    │     ├── Fetch OHLCV (15m, 1h, 4h)
                    │     ├── Compute features
                    │     ├── HMM predict → (regime, confidence)
                    │     ├── Compute conviction score (8 factors)
                    │     ├── If score ≥ 40 and no open position: open trade
                    │     └── If open position: check SL/TP/T1/T2/trailing
                    └── Sync tradebook → write engine_state.json
```

Key Methods

Method	Purpose
`run()`	Entry point — starts the bot loop
`_heartbeat()`	30-second cycle — read commands, update state
`_analysis_cycle()`	5-minute cycle — full HMM scan across all coins
`_analyze_coin(symbol)`	Per-coin regime detection + conviction scoring
`_manage_position(symbol)`	Monitor open position: SL/TP/T1/T2 checks
`_process_commands()`	Handle dashboard commands (start, stop, pause, close-all)
`_update_engine_state()`	Write engine health to data/engine_state.json

Bot States

```
IDLE → RUNNING → PAUSED → RUNNING → STOPPED
               ↓
           KILLED (kill switch triggered)
```

3.2 Data Pipeline

File: `data_pipeline.py` (8.2 KB)

Responsible for all market data ingestion.

Data Sources

Source	Purpose	Fallback
Binance Futures	OHLCV for all timeframes (primary)	CoinDCX
CoinDCX	Live spot/futures prices	Binance
Binance Spot	Funding rate proxy (perpetual funding)	—

Timeframes Used

Timeframe	Purpose	Candles
`15m`	Entry/exit timing, HMM training data	250 candles
`1h`	Trend confirmation	100 candles
`4h`	Macro regime, S/R lookback	150 candles
`1d`	Long-term trend context	60 candles

Key Function

```python
def fetch_ohlcv(symbol, interval, limit=250) -> pd.DataFrame:
    # Returns: open, high, low, close, volume columns
```

```
    # Tries Binance first, falls back to CoinDCX
    # Normalizes column names and datetime index
```

3.3 Feature Engine

File: `feature_engine.py` (11.6 KB)

Computes all technical indicators and the 6 HMM input features.

HMM Feature Set (6 Features)

These 6 features are the inputs to the Gaussian HMM. Selected via greedy ablation — adding `funding_proxy` and `adx` boosted Sharpe from 0.258 → 0.667.

Feature	Formula	Why It Matters
`log_return`	`ln(close_t / close_{t-1})`	Raw price momentum signal
`volatility`	Rolling 20-period std of log_return	Regime separator (low in BULL, high in BEAR)
`volume_change`	`(vol_t - vol_{t-1}) / vol_{t-1}`	Volume expansion confirms breakouts
`rsi_norm`	`(RSI_14 - 50) / 50`	Normalized [-1, +1] momentum indicator
`funding_proxy`	`close / EMA(close, 20) - 1`	Proxy for futures funding sentiment
`adx`	Average Directional Index (14)	Trend strength (separates trend from chop)

Additional Technical Indicators (for Conviction Scoring)

Indicator	Purpose
ATR (14)	Stop-loss/take-profit placement
VWAP	Intraday fair value reference
EMA (20, 50, 200)	Trend direction across timeframes
Bollinger Bands (20, 2σ)	Sideways/CHOP strategy entry
Support/Resistance Zones	Price levels using pivot point method
RSI divergence	Momentum vs price mismatch

Key Functions

```python
def compute_hmm_features(df) -> pd.DataFrame:
    # Returns df with HMM_FEATURES columns added

def compute_atr(df, period=14) -> pd.Series:
    # True Range rolling mean

def compute_vwap(df) -> pd.Series:
    # (price × volume).cumsum() / volume.cumsum()
```

```
def compute_sr_position(df_4h, lookback=50) -> float:
    # Returns position relative to S/R: -1 (at support) to +1 (at resistance)
```

3.4 HMM Brain

File: `hmm_brain.py` (8.5 KB)

The core intelligence module. Wraps `hmmlearn.GaussianHMM` with regime labeling and margin confidence.

Model Parameters

Parameter	Value	Reason
`n_states`	3	BULL / CHOP / BEAR (4-state tried: Sharpe 0.72 vs 1.22 for 3-state)
`covariance_type`	`full`	Captures cross-feature correlations
`n_iter`	100	EM convergence iterations
`lookback`	250 candles	~2.6 days of 15m data
`retrain_every`	24 hours	Adapts to new market conditions

Why CRASH State Was Removed

When 4-state HMM was tested, the CRASH state achieved only **10.9% directional accuracy** — worse than random (25% for 4 states). The 3-state model (Sharpe 1.22) significantly outperforms 4-state (Sharpe 0.72). CRASH events are now classified as extreme BEAR.

State Labeling Algorithm

After training, raw HMM states have no semantic meaning (they're just 0, 1, 2). The `_build_state_map()` method re-labels them by sorting on mean log-return:

```
sorted_indices = np.argsort(means)[::-1]
# sorted_indices[0] = highest mean log-return → BULL
# sorted_indices[1] = middle return           → CHOP
# sorted_indices[2] = lowest mean log-return → BEAR
```

Confidence: Margin vs Raw Posterior

Problem with raw posterior: `model.predict_proba()` returned values of 99%+ for nearly every prediction regardless of actual accuracy. This made it completely uncalibrated and useless as a confidence signal.

Solution — Margin confidence:

```
sorted_p = np.sort(probs)[::-1]
confidence = sorted_p[0] - sorted_p[1]  # best - 2nd_best
```

Margin	Interpretation
0.0 – 0.10	Very uncertain (below LOW threshold → no trade)

0.10 – 0.25	Low confidence
0.25 – 0.40	Medium confidence
0.40 – 0.60	Medium-high confidence
0.60 – 1.00	High confidence

Margin Confidence Tiers

```
HMM_CONF_TIER thresholds:
  HIGH      = 0.60   → full HMM score (44 pts)
  MED_HIGH  = 0.40   → ~33 pts
  MED       = 0.25   → ~22 pts
  LOW       = 0.10   → ~11 pts
  < 0.10            → 0 pts (HMM untrusted)
```

Retraining Schedule

The model retrains every 24 hours (`HMM_RETRAIN_HOURS = 24`) on the latest 250 candles of 15m OHLCV. `needs_retrain()` checks `datetime.utcnow() - last_trained > 24h` .

3.5 Coin Scanner

File: `coin_scanner.py` (10.3 KB)

Dynamically selects which coins to trade each cycle.

Coin Tier Classification

Tiers determined by backtesting Sharpe ratio on forward-test windows (from `tools/experiment_3state_calibration.py`). Stored in `data/coin_tiers.csv` .

Tier	Criteria	Coins
A (Trade)	Forward-test Sharpe ≥ 1.0	ENA, SEI, ZEC, PHA, CHZ, PEPE, ADA, XRP, TIA, ETHFI, DOGE, AAVE, WLD, AR, ETC, BCH, FET, KAVA
B (Monitor)	Sharpe 0.5–1.0	BTC, ETH, SOL (and others)
C (Avoid)	Sharpe < 0.5 or volatile	CAKE, FXS, DOT, XLM, AVAX, SHIB, TAO, WIF, AUDIO, GALA, OP, INJ, SUI, NEAR, BNB, BONK, LTC

Scan Process

```
1. Fetch top 50 coins by 24h USDT volume from Binance
2. Filter out Tier C coins
3. Sort: Tier A coins first, then Tier B by volume
4. Return top N = TOP_COINS_LIMIT (25) for analysis
5. Re-scan every SCAN_INTERVAL_CYCLES = 4 (every 20 minutes)
```

3.6 Risk Manager

File: `risk_manager.py` (18.2 KB)

Enforces all risk rules. The "Anti-Liquidation" module.

Conviction Score Formula

The conviction score (0–100) gates every trade and determines leverage:

```
Score = HMM(44) + BTC_macro(7) + Funding(11) + SR_VWAP(2)
      + OI(11) + Vol(0) + Sentiment(15) + OrderFlow(10)
      = 100 pts maximum
```

Factor	Max Points	What It Measures
HMM Regime	44	Regime type + margin confidence tier
Sentiment	15	VADER/FinBERT + CryptoPanic + Fear & Greed score
Funding Rate	11	Funding rate direction + magnitude (futures sentiment)
Open Interest	11	OI trend — rising OI confirms new money in direction
Order Flow	10	Taker buy/sell imbalance + L2 depth ratio
BTC Macro	7	BTC correlation (coin moves with BTC regime)
S/R + VWAP	2	Entry near support (long) or resistance (short)
Volatility	0	Filter only — blocks trade if ATR% out of range

HMM Conviction Points Breakdown

Regime	Margin Tier	Points
BULL / BEAR	HIGH (≥0.60)	44
BULL / BEAR	MED_HIGH (≥0.40)	33
BULL / BEAR	MED (≥0.25)	22
BULL / BEAR	LOW (≥0.10)	11
CHOP	Any	0 (no trend trade)
Any	< 0.10	0 (model untrusted)

Leverage Bands

```
Conviction Score → Leverage
< 40             → 0x   (no trade)
40 – 54          → 10x
55 – 69          → 15x
70 – 84          → 25x
85 – 100         → 35x
```

Position Sizing Formula

```
risk_amount = balance × RISK_PER_TRADE (4%)
stop_distance = ATR × sl_multiplier(leverage)
quantity = risk_amount / stop_distance
max_qty = (balance × leverage) / entry_price
final_qty = min(quantity, max_qty)
```

ATR Multipliers by Leverage

Higher leverage → tighter stops to maintain consistent portfolio risk:

Leverage	SL Multiplier	TP Multiplier	Risk:Reward
1–4x	1.5	3.0	1:2
5–9x	1.2	2.4	1:2
10–24x	1.0	2.0	1:2
25–49x	0.7	1.4	1:2
≥ 50x	0.5	1.0	1:2

Kill Switch

```python
KILL_SWITCH_DRAWDOWN = 0.10  # 10% portfolio loss in 24h → bot stops

if drawdown_24h >= 0.10:
    self._killed = True
    close_all_positions()
    send_telegram_alert("KILL SWITCH TRIGGERED")
```

Volatility Filter

```python
VOL_MIN_ATR_PCT = 0.003  # 0.3% minimum ATR (too quiet = no trade)
VOL_MAX_ATR_PCT = 0.06   # 6.0% maximum ATR (too chaotic = no trade)
```

3.7 Execution Engine

File: `execution_engine.py` (21.7 KB)

Handles all order placement — paper (simulation) and live (real exchange).

Trading Modes

Mode	Exchange	Real Money	Purpose
Paper	Binance Testnet	No	Safe testing, SaaS user copy-trades
Live	CoinDCX	Yes	Real margin futures (admin/advanced users)

Multi-Target Profit Booking (T1/T2/T3)

The primary exit strategy. Configured in `config.py` :

```
MT_RR_RATIO = 5         (SL : T3 = 1:5)
MT_T1_FRAC  = 0.333     T1 at 33.3% of T3 distance
MT_T2_FRAC  = 0.666     T2 at 66.6% of T3 distance
MT_T1_BOOK_PCT = 0.25   Book 25% of qty at T1
MT_T2_BOOK_PCT = 0.50   Book 50% of remaining qty at T2
```

Example (LONG @ $100, ATR = $2, SL mult = 1.0 at 10x leverage):

```
Entry  = $100.00
SL     = $98.00  (entry - 1×ATR)
T1     = $103.33  (entry + 1.667×ATR)  → book 25%
T2     = $106.67  (entry + 3.333×ATR)  → book 50% of remaining
T3     = $110.00  (entry + 5.0×ATR)    → close remainder
```

PnL per stage at 10x leverage on $100 capital:

- T1 hit: +$8.33 profit on 25% of position
- T2 hit: +$16.67 profit on 37.5% of position
- T3 hit: +$25.00 profit on 37.5% of position
- Max T3 PnL (all targets hit): +$16.67 per $100 capital

Trailing Stop-Loss

```
TRAILING_SL_ENABLED = True
TRAILING_SL_ACTIVATION_ATR = 1.0  # Activates when price moves 1×ATR in favor
TRAILING_SL_DISTANCE_ATR = 1.0    # SL stays 1×ATR behind price peak

# Logic:
if price_move_favor >= 1.0 × ATR:
    trail_active = True
    new_sl = peak_price - 1.0 × ATR  # trails peak, never goes backward
```

Order Flow

```
open_position()
  ├── Validate conviction score ≥ 40
  ├── Calculate position size (2% risk rule)
  ├── Set SL, T1, T2, T3 levels
  ├── Place market order (paper: simulate, live: CoinDCX REST API)
  └── Log to tradebook.json

close_position(reason)
  ├── Place market close order
  ├── Record exit price, PnL, exit reason
  ├── Append PartialBooking records for T1/T2
  └── Mark trade as "closed" in tradebook
```

3.8 Order Flow Engine

File: `orderflow_engine.py` (37.9 KB)

Analyzes microstructure signals from the order book.

Signals Computed

Signal	Method	Conviction Impact
L2 Depth Imbalance	(bid_vol - ask_vol) / (bid_vol + ask_vol)	Directional pressure
Taker Buy/Sell Flow	Aggressor-side trade flow from recent trades	Short-term momentum
Cumulative Delta	Running sum of (buy_vol - sell_vol)	Sustained buying/selling pressure
Weighted Price Levels	Volume-weighted bid/ask clusters	Key price magnets

These feed into the **Order Flow factor (10 pts)** of the conviction score.

3.9 Sentiment Engine

Files: `sentiment_engine.py` (21.1 KB), `sentiment_sources.py` (18.5 KB)

Data Sources

Source	Weight	API/Method
CryptoPanic	Primary news	REST API (free key, rate limited)
RSS Feeds	5 outlets (CoinDesk, CoinTelegraph, Decrypt, The Block, Blockworks)	`feedparser` library
Reddit	r/CryptoCurrency, coin-specific subs	PRAW (OAuth) or public JSON
Fear & Greed Index	Macro sentiment	alternative.me API

NLP Pipeline

```
Raw text → VADER (always, fast) → score [-1, +1]
         → FinBERT (optional, lazy-load, ~400MB PyTorch model)
              → probability for [positive, negative, neutral]
         → Weighted average → final_score

Buzz = article count in last 4 hours (SENTIMENT_WINDOW_HOURS)
Momentum = change in score from previous period
```

Output: SentimentSignal

```python
@dataclass
class SentimentSignal:
    score: float        # -1.0 (very bearish) to +1.0 (very bullish)
    confidence: float   # 0.0 to 1.0 (based on article count)
    buzz: int           # number of relevant articles
    momentum: float     # score change vs last period
    alert: bool         # True = hack/exploit detected → hard veto (conviction = 0)
```

Integration in Conviction Score

```python
# Sentiment contributes up to 15 pts
if alert:
    conviction_score = 0  # Hard veto — overrides everything
else:
    sentiment_pts = normalize(score) × 15
```

3.10 Tradebook

File: `tradebook.py` (39.3 KB)

Persistent trade ledger backed by `data/tradebook.json`.

Trade Lifecycle

```
OPEN → [T1_HIT] → [T2_HIT] → CLOSED
     → SL_HIT   → CLOSED
     → MANUAL    → CLOSED
     → KILL_SW   → CLOSED
```

Data Stored Per Trade

```json
{
    "trade_id": "T-0042-DOGEUSDT",
    "symbol": "DOGEUSDT",
    "side": "long",
    "regime": "BULLISH",
    "confidence": 0.73,
    "conviction_score": 82,
    "leverage": 25,
    "capital": 100.0,
    "entry_price": 0.1234,
    "current_price": 0.1267,
    "sl": 0.1215,
    "t1": 0.1256, "t2": 0.1278, "t3": 0.1300,
    "t1_hit": True, "t2_hit": False,
    "trailing_sl": 0.1245, "trailing_active": True,
    "pnl": 5.34,
    "status": "active",
```

```
    "entry_time": "2026-03-07T10:00:00Z"
}
```

3.11 Engine API

File: `engine_api.py` (8.8 KB)

Flask REST server that bridges Python engine ↔ Next.js dashboard.

Endpoints

Method	Endpoint	Returns
GET	`/api/health`	Engine status, uptime, cycle count
GET	`/api/bot-state`	All coin regimes, active positions, engine state
GET	`/api/trades`	All trades from tradebook.json
POST	`/api/close-trade`	Close a specific trade by trade_id or symbol
POST	`/api/close-all`	Close all open positions (KILL_ALL command)

---

4. Brain Logic — Deep Dive

Why Hidden Markov Model?

Markets cycle through regimes (trending, ranging, volatile). Unlike indicators that react to price, an HMM learns the **hidden state** (regime) from observable features. Key advantages:

1. **Probabilistic** — outputs confidence, not binary signal
2. **Temporal** — considers sequences of observations, not just current bar
3. **Unsupervised** — discovers regimes from data without labeling

Feature Selection Rationale

Experiment: Greedy Feature Addition Starting from 4 features (log_return, volatility, volume_change, rsi_norm):

Configuration	Sharpe	Notes
4 features (baseline)	0.258	Too little context
+ funding_proxy	0.481	Futures sentiment adds edge
+ adx (6 features)	0.667	Trend strength resolves CHOP ambiguity
+ extra features	< 0.667	Overfitting / curse of dimensionality

Final: 6 features selected (confirmed Mar 2026)

3-State vs 4-State Comparison

Metric	3-State	4-State

Sharpe Ratio	**1.22**	0.72
Directional Accuracy	**35.5%**	29.0%
CRASH state accuracy	—	10.9%
Verdict	✅ Production	❌ Removed

Regime Trading Rules

Regime	Trade Direction	Strategy
BULL	Long only	Trend following
BEAR	Short only	Trend following
CHOP	Sideways	Mean reversion (Bollinger Bands, RSI)

In CHOP, the `sideways_strategy.py` module takes over with:

- Bollinger Band entries (price below lower band → long, above upper → short)
- RSI extremes (< 35 = oversold → long; > 65 = overbought → short)
- 30% smaller position size (`SIDEWAYS_POSITION_REDUCTION = 0.30`)

Support/Resistance Experiment Results

Experiment file: `tools/experiment_sr_4h_ic.py`

Metric	Result	Interpretation
IC (Information Coefficient)	-0.010	Near zero — weak predictive value
t-statistic	-0.81	Not statistically significant (need
Best lookback tested	150×4h (25 days)	Sharpe 0.818 but t-stat weak
Decision	Keep weight = 2 pts	Too minor to remove, not enough to increase

---

5. Risk Management System

Five-Layer Risk Hierarchy

```
Layer 1: Conviction Gating    → Score < 40 = NO TRADE
Layer 2: Leverage Bands       → Score → 0x / 10x / 15x / 25x / 35x
Layer 3: Position Sizing      → 4% risk per trade (ATR-based)
Layer 4: Trade-Level Stops    → SL + T1/T2/T3 + trailing SL
Layer 5: Portfolio Kill Switch→ 10% drawdown in 24h → stop all
```

Trade Execution Decision Tree

```
Coin available?
│
├─ No → Skip
```

```
└─ Yes
     │
     ├─ Already have open position?
     │    └─ Yes → Monitor SL/TP, skip new entry
     │
     └─ No open position
          │
          ├─ Fetch OHLCV → Compute features → HMM predict
          │
          ├─ Regime == CHOP? → Use sideways_strategy
          │
          ├─ Compute conviction score (8 factors)
          │
          ├─ Score < 40? → SKIP (no trade)
          │
          ├─ Volatility filter pass? → ATR% in [0.3%, 6%]
          │
          ├─ Kill switch active? → BLOCK all trades
          │
          └─ OPEN TRADE
               ├─ Calculate leverage band (10x / 15x / 25x / 35x)
               ├─ Size position (4% risk rule)
               ├─ Set SL (ATR × sl_mult)
               ├─ Set T1, T2, T3 targets
               └─ Place order (paper or live)
```

Exit Logic

```
Every 30-second heartbeat:
  For each open position:
    ├─ price ≤ SL?           → CLOSE (stop_loss)
    ├─ price ≥ T1 (not hit)? → Book 25%, set SL to breakeven
    ├─ price ≥ T2 (not hit)? → Book 50% of remaining
    ├─ price ≥ T3?           → Close remainder (full target)
    ├─ trailing_sl active?   → Update SL = peak − 1×ATR
    ├─ pnl < MAX_LOSS_PCT?   → CLOSE (max_loss −15% per trade)
    └─ hold < MIN_HOLD_MIN?  → Do not exit on regime flip alone (30 min hold)
```

---

6. SaaS Dashboard

6.1 Technology Stack

Layer	Technology	Version
Framework	Next.js (App Router)	14.2.35
Language	TypeScript	5.2.2
UI	React + Tailwind CSS + shadcn/ui	18.2.0
ORM	Prisma	6.7.0

Database	PostgreSQL	(Railway managed)
Auth	NextAuth.js	4.24.11
Animations	Framer Motion	10.18.0
Charts	Recharts	3.7.0
Icons	Lucide React	0.446.0
Payments	Razorpay	webhook-based
State	Zustand	5.0.3
Notifications	Sonner (toast)	1.5.0

6.2 Application Pages

Public Pages

Route	File	Purpose
`/`	`app/page.tsx`	Landing page (hero, features, pricing CTA)
`/login`	`app/login/page.tsx`	User login (email/password, NextAuth)
`/signup`	`app/signup/page.tsx`	Registration with tier selection
`/pricing`	`app/pricing/page.tsx`	Plan comparison (Free / Pro / Ultra)
`/howto`	`app/howto/page.tsx`	User guide, getting started

Protected Pages (auth required)

Route	File	Purpose
`/dashboard`	`app/dashboard/`	Main control panel: HMM regimes, live positions, metrics
`/bots`	`app/bots/`	Create/manage bot instances, start/stop
`/trades`	`app/trades/`	Trade history, open trades, P&L breakdown
`/performance`	`app/performance/`	Session analytics: ROI, Sharpe, drawdown, win rate
`/intelligence`	`app/intelligence/`	Sentiment signals, order flow data
`/account`	`app/account/`	Profile, exchange API keys, subscription info

Admin Pages (role=admin only)

Route	Component	Purpose
`/admin`	`admin-client.tsx`	Full admin control panel
└ User Analytics	`user-analytics.tsx`	Signups, churn, active users
└ Revenue Dashboard	`revenue-dashboard.tsx`	ARR, ARPU, subscription metrics

└ Subscription Mgmt	`subscription-mgmt.tsx`	Override user tiers, Razorpay reconciliation
└ Engine Control	`engine-control.tsx`	Start/stop bot engine globally
└ System Health	`system-health.tsx`	CPU, memory, disk, uptime
└ Audit Log	`audit-log.tsx`	API calls, user actions

6.3 API Routes

All routes in `sentinel-saas/nextjs_space/app/api/` :

Bot Management

Method	Route	Description
POST	`/api/bots/create`	Create new bot for authenticated user
POST	`/api/bots/delete`	Delete bot and its trades
POST	`/api/bots/toggle`	Start (creates BotSession) or Stop bot
POST	`/api/bots/kill`	Emergency kill — closes all paper trades immediately
GET/POST	`/api/bots/config`	Read/write bot configuration
GET	`/api/bots/logs`	Fetch bot activity logs
GET	`/api/bot-state`	Live bot state (regimes, positions, cycle info)

Trade Management

Method	Route	Description
GET	`/api/trades`	All trades for user (syncs from engine)
POST	`/api/trades/close`	Manually close a specific trade
POST	`/api/reset-trades`	Clear all trades (admin/test only)

Session Management

Method	Route	Description
GET	`/api/sessions`	List all bot sessions with metrics
POST	`/api/sessions/backfill`	Admin: tag legacy trades as Session 0

Exchange & Wallet

Method	Route	Description
POST	`/api/exchange/validate`	Test API credentials (no storage)
GET	`/api/exchange/positions`	Live positions from exchange

GET	`/api/wallet-balance`	Account balance using stored keys
GET/POST	`/api/settings/api-keys`	Read/write encrypted API keys

User & Subscription

Method	Route	Description
POST	`/api/signup`	Register new user
GET	`/api/subscription/status`	Current tier, trial status, expiry
POST	`/api/subscription/update`	Upgrade/downgrade plan
POST	`/api/webhooks/razorpay`	Razorpay payment confirmed → activate tier

System

Method	Route	Description
GET	`/api/health`	API health check
GET	`/api/live-market`	Live price feed
GET	`/api/debug`	Debug info (engine PID, state dump)

6.4 Database Schema

12 Models in PostgreSQL via Prisma ORM.

Entity Relationship Overview

```
User
  ├── Subscription (1:1)
  ├── ExchangeApiKey[] (1:many — one per exchange)
  ├── Bot[] (1:many)
  │     ├── BotConfig (1:1)
  │     ├── BotState (1:1)
  │     ├── BotSession[] (1:many)
  │     └── Trade[] (1:many)
  │           └── PartialBooking[] (1:many)
  ├── Account[] (NextAuth OAuth)
  └── Session[] (NextAuth sessions)
```

Model Details

User

```
id, email (unique), name, password (hashed), role (user|admin)
referralCode, phone, createdAt, updatedAt
```

Subscription

```
id, userId (unique), tier (free|pro|ultra), status (trial|active|expired)
coinScans (0=unlimited), trialEndsAt, currentPeriodEnd
razorpayPaymentId, razorpayOrderId
```

ExchangeApiKey

```
id, userId, exchange (binance|coindcx)
apiKey (AES-256-GCM encrypted), apiSecret (AES-256-GCM encrypted)
encryptionIv (comma-separated key IV + secret IV)
isActive, createdAt, updatedAt
UNIQUE: (userId, exchange)
```

Bot

```
id, userId, name, exchange, status (running|stopped|paused|error)
isActive, startedAt, stoppedAt, createdAt, updatedAt
```

BotConfig

```
id, botId (unique), mode (paper|live)
capitalPerTrade, maxOpenTrades, slMultiplier, tpMultiplier, maxLossPct
multiTargetEnabled, t1/t2/t3Multiplier, t1/t2BookPct
coinList (JSON array), leverageTiers (JSON)
```

BotState

```
id, botId (unique), engineStatus (idle|running|error)
lastCycleAt, cycleCount, cycleDurationMs
coinStates (JSON: {SYMBOL: {regime, features, signal}})
errorMessage, errorAt, updatedAt
```

Trade

```
id, botId, coin, position (long|short), regime, confidence, mode (paper|live)
leverage, capital, quantity, entryPrice, currentPrice, exitPrice
stopLoss, takeProfit, t1/t2/t3Price, t1/t2Hit (bool)
trailingSl, trailingTp, trailingActive, trailSlCount
activePnl, activePnlPercent, totalPnl, totalPnlPercent
status (active|closed|cancelled), exitReason, exitPercent
exchangeOrderId, exchangePositionId, sessionId
entryTime, exitTime, createdAt, updatedAt
```

BotSession

```
id, botId, sessionIndex (0=legacy, 1=first run...)
startedAt, endedAt, status (active|closed), mode (paper|live)
totalTrades, closedTrades, winTrades
totalPnl, roi (totalPnl/totalCapital×100), winRate (winTrades/closedTrades×100)
maxDrawdown, bestTrade, worstTrade, totalCapital
createdAt, updatedAt
```

PartialBooking

```
id, tradeId, target (T1|T2|T3)
bookPercent, quantity, exitPrice, pnl, pnlPercent
createdAt
```

6.5 Authentication & Authorization

Provider: NextAuth.js v4 with JWT strategy.

Auth Flow

```
User submits email + password
  → /api/auth/[...nextauth]
  → CredentialsProvider validates against Prisma User (bcrypt compare)
  → JWT created with {id, email, name, role}
  → Session cookie set (httpOnly, secure)
  → Middleware checks session on every protected route
```

Authorization Levels

Role	Access
`user`	Own bots, trades, sessions, account settings
`admin`	All users' data, engine control, subscription management

Session in API Routes

```
const session = await getServerSession(authOptions);
if (!session?.user) return 401;
const userId = (session.user as any).id;
const isAdmin = (session.user as any).role === 'admin';
// isAdmin bypasses userId filter — can see all data
```

6.6 Subscription System

Tiers

Tier	Coin Scans	Price	Features
Free	0 (trial)	$0	7-day trial, limited coins
Pro	15	$X/mo	Multi-coin, paper trading
Ultra	50 (unlimited)	$XX/mo	All features, live trading

Payment Flow (Razorpay)

```
User clicks Upgrade → /pricing
  → Creates Razorpay order → returns order_id
  → User completes payment in Razorpay checkout
  → Razorpay sends webhook to /api/webhooks/razorpay
```

```
  → Verify signature (HMAC-SHA256)
  → Update subscription.tier + status + currentPeriodEnd in Prisma
  → Send confirmation email
```

6.7 Exchange API Key Management

Security Model

API keys are never stored in plaintext. Full encryption pipeline:

```
User enters key + secret in browser
  → POST /api/settings/api-keys (HTTPS)
  → Server: encryptApiKeys(key, secret) using AES-256-GCM
      ├── Generate random 16-byte IV for key
      ├── Generate random 16-byte IV for secret
      ├── Encrypt: ciphertext + auth_tag
      └── Store: {apiKey, apiSecret, encryptionIv} in ExchangeApiKey table

Decryption (for wallet-balance / exchange validation):
  → Fetch ExchangeApiKey from DB
  → decryptApiKeys(encApiKey, encApiSecret, encryptionIv)
  → Use decrypted keys for single API call
  → Never log or return raw keys to client
```

ENCRYPTION_KEY must be set as a 64-character hex string in Railway env vars. Never change it after keys are stored — all stored keys become unreadable.

6.8 Bot Session Lifecycle

A **BotSession** captures one complete bot run (start → stop) as a discrete record with aggregated metrics.

Session States

```
Bot START button clicked
  → createBotSession(botId, mode)
  → BotSession: {status: "active", sessionIndex: N, startedAt: now}
  → syncEngineTrades() tags new trades with sessionId

Bot STOP button clicked
  → closeBotSession(botId)
  → Close open paper trades: {status: "closed", exitReason: "BOT_STOPPED"}
  → For live trades: POST ENGINE_API_URL/api/close-all
  → Compute metrics: totalPnl, roi, winRate, maxDrawdown, bestTrade, worstTrade
  → BotSession: {status: "closed", endedAt: now, ...metrics}
```

Session Metrics Formula

```
ROI (%) = totalPnl / totalCapital × 100
totalCapital = Σ(trade.capital) for all trades in session
winRate (%) = winTrades / closedTrades × 100
winTrades = trades where totalPnl > 0
```

Legacy Session (Session 0)

For trades created before session tracking was introduced:

```
POST /api/sessions/backfill (admin only)
  → Creates BotSession {sessionIndex: 0, status: "closed"}
  → Tags all untagged trades with this sessionId
  → Computes metrics from historical trade data
```

---

7. Data Flow & Integration

Trade Sync: Engine → Dashboard

The Python engine writes to `tradebook.json`. The Next.js dashboard syncs this into PostgreSQL on every `/api/trades` request:

```
1. GET /api/trades (Next.js API route)
   │
   ├─ Find userBot in Prisma (Bot where userId = session.user.id)
   │
   ├─ If userBot.startedAt is set:
   │    │
   │    ├─ Fetch engine trades from ENGINE_API_URL/api/trades
   │    │
   │    └─ syncEngineTrades(engineTrades, botId, botStartedAt)
   │          ├─ DELETE trades where entryTime < botStartedAt (purge stale)
   │          └─ UPSERT each engine trade into Prisma Trade table
   │               └─ Tag with sessionId = active BotSession
   │
   └─ Return trades from Prisma (source of truth)
```

Live Bot State: Engine → Dashboard

```
Dashboard polls GET /api/bot-state every 10 seconds
  → Fetches from ENGINE_API_URL/api/bot-state
  → Falls back to Prisma BotState if engine unreachable
  → Returns: {coinStates, activeTrades, engineStatus, lastCycleAt}
```

Commands: Dashboard → Engine

```
User clicks START / STOP / KILL in dashboard
  → POST /api/bots/toggle or /api/bots/kill
  → Writes JSON to data/commands.json:
      {"command": "STOP", "timestamp": "2026-03-07T10:00:00+05:30"}
  → Python bot reads commands.json on every 30-second heartbeat
  → Executes command: start analysis, stop loop, close positions
```

---

8. Backtesting & Experimentation

Methodology

All backtests use **walk-forward validation** (not in-sample):

- Train on 80% of data
- Test on remaining 20%
- Roll forward window
- Report average Sharpe, IC, win rate across all windows

Experiment Results Summary

Experiment	File	Key Finding
3-state calibration	`experiment_3state_calibration.py`	3-state Sharpe 1.22 > 4-state 0.72
Feature selection	`experiment_features_weights.py`	6-feature set optimal (Sharpe 0.667)
S/R 4h IC test	`experiment_sr_4h_ic.py`	IC = -0.010, t = -0.81 (not significant)
100-coin evaluation	`backtest_fronttest_100coins.py`	Generated coin_tiers.csv
Model comparison	`backtest_compare.py`	v1 vs v2 head-to-head

Coin Tier Generation

```
tools/experiment_3state_calibration.py
  → Fetches 1 year of OHLCV for 100 coins
  → Runs HMM + conviction system on each
  → Walk-forward Sharpe per coin
  → Outputs data/coin_tiers.csv
       Tier A: Sharpe ≥ 1.0
       Tier B: Sharpe 0.5–1.0
       Tier C: Sharpe < 0.5
```

Running a Backtest

```
# Historical backtest (single coin, 1 year)
python tools/backtest_historical.py --coin BTC --days 365

# 100-coin walk-forward evaluation
python tools/backtest_fronttest_100coins.py

# Compare two model versions
python tools/backtest_compare.py --v1 baseline --v2 6features

# Sentiment backtester
python sentiment_backtester.py --coin BTC --days 30
```

---

9. Deployment & Infrastructure

Docker Build

Multi-stage build combining Python 3.11 + Node.js 20:

```
Stage 1 (deps): Install Python requirements + Node modules
Stage 2 (builder): npx prisma generate + next build
Stage 3 (runner): Copy built app, expose port 3000

Startup: start.sh
  → npx next start -p 3000  (dashboard)
  → Python bot spawned via dashboard UI (not started at boot)
```

Railway Deployment

```
Service: sentinel-saas (Next.js + Python)
  Port: 3000 (Next.js)
  Build: Dockerfile in sentinel-saas/nextjs_space/
  Start: npx next start -p 8080

Database: Railway PostgreSQL plugin
  → DATABASE_URL automatically injected

Auto-deploy: On every git push to main branch
```

Required Environment Variables

Variable	Purpose	Where to Set
`DATABASE_URL`	PostgreSQL connection string	Railway (auto-set)
`NEXTAUTH_SECRET`	JWT signing key (random 32+ chars)	Railway
`NEXTAUTH_URL`	App URL (https://your-app.railway.app)	Railway
`ENCRYPTION_KEY`	AES-256-GCM key for API key storage (64-char hex)	Railway
`ENGINE_API_URL`	URL to Python Flask API	Railway
`BINANCE_API_KEY`	Binance API (paper trade)	Railway
`BINANCE_API_SECRET`	Binance API secret	Railway
`COINDCX_API_KEY`	CoinDCX API (live trade)	Railway
`COINDCX_API_SECRET`	CoinDCX API secret	Railway
`TELEGRAM_BOT_TOKEN`	Telegram alerts	Railway
`TELEGRAM_CHAT_ID`	Telegram chat ID	Railway
`RAZORPAY_KEY_ID`	Payment gateway	Railway
`RAZORPAY_KEY_SECRET`	Payment gateway secret	Railway
`RAZORPAY_WEBHOOK_SECRET`	Webhook signature validation	Railway

DigitalOcean Alternative (DEPLOY.md)

```
# Ubuntu 24.04 Droplet
apt install docker.io docker-compose nginx certbot

# Clone repo + set .env
docker-compose up -d

# SSL with Let's Encrypt
certbot --nginx -d yourdomain.com
```

Database Migrations

```
cd sentinel-saas/nextjs_space

# Development (destructive reset allowed)
npx prisma db push --accept-data-loss

# Production (tracked migrations)
npx prisma migrate dev --name add_bot_sessions
npx prisma generate
```

---

10. Testing

Unit Test Suite

File: `tests/test_unit.py` **Total:** 140 tests — all passing **Policy:** No real API calls — all external dependencies mocked

```
python -m pytest tests/test_unit.py -v --tb=short
```

Test Coverage by Module

Class	Tests	Coverage
`TestConfigNewConstants`	9	Config values, env var loading
`TestRiskManagerScoring`	50	Conviction formula, all 8 factors, leverage bands
`TestExecutionEngineUnit`	16	Order logic, multi-target booking, trailing SL
`TestDataPipelineUnit`	7	OHLCV fetch (mocked), fallback logic
`TestSentimentSourcesUnit`	6	API mocking (CryptoPanic, RSS, Fear & Greed)
`TestFeatureEngineUnit`	8	RSI, ATR, VWAP calculations
`TestSentimentEngineUnit`	8	Sentiment aggregation, VADER integration
`TestHMMBrainUnit`	16	Training, regime prediction, confidence, retrain

`TestCoinScannerUnit`	7	Tier filtering, coin ordering

---

11. Configuration Reference

Key Config Values (`config.py`)

HMM

```
HMM_N_STATES = 3           # Bull, Chop, Bear
HMM_COVARIANCE = "full"    # Full covariance matrix
HMM_ITERATIONS = 100       # EM iterations
HMM_LOOKBACK = 250         # Training candles (15m)
HMM_RETRAIN_HOURS = 24     # Retrain frequency
```

Bot Loop

```
LOOP_INTERVAL_SECONDS = 30      # Heartbeat
ANALYSIS_INTERVAL_SECONDS = 300 # Full cycle (5 min)
ERROR_RETRY_SECONDS = 60        # Error retry
```

Multi-Coin

```
MAX_CONCURRENT_POSITIONS = 15   # Max open trades
TOP_COINS_LIMIT = 25            # Coins to scan
CAPITAL_PER_COIN_PCT = 0.05     # 5% of balance per coin
SCAN_INTERVAL_CYCLES = 4        # Re-scan every 20 min
```

Risk

```
RISK_PER_TRADE = 0.04           # 4% balance at risk per trade
KILL_SWITCH_DRAWDOWN = 0.10     # 10% 24h drawdown → stop
MAX_LOSS_PER_TRADE_PCT = -15    # Hard max-loss per trade
MIN_HOLD_MINUTES = 30           # Min hold before regime exit
```

Multi-Target

```
MT_RR_RATIO = 5                 # Risk : T3 = 1:5
MT_T1_FRAC = 0.333              # T1 at 33.3% of T3 distance
MT_T2_FRAC = 0.666              # T2 at 66.6% of T3 distance
MT_T1_BOOK_PCT = 0.25           # Book 25% at T1
MT_T2_BOOK_PCT = 0.50           # Book 50% of remaining at T2
```

Trailing SL

```
TRAILING_SL_ENABLED = True
TRAILING_SL_ACTIVATION_ATR = 1.0  # Activate after 1×ATR move in favor
```

```
TRAILING_SL_DISTANCE_ATR = 1.0    # Trail 1×ATR behind peak
```

Sentiment

```
SENTIMENT_ENABLED = True
SENTIMENT_CACHE_MINUTES = 15   # Cache per-coin results
SENTIMENT_WINDOW_HOURS = 4     # Lookback window
SENTIMENT_MIN_ARTICLES = 3     # Min articles for valid score
```

---

12. Glossary

Term	Definition
HMM	Hidden Markov Model — probabilistic sequence model for regime classification
Gaussian HMM	HMM where observations follow Gaussian (normal) distributions per state
Regime	Market state: BULL (uptrend), CHOP (sideways), BEAR (downtrend)
Margin Confidence	`best_prob - 2nd_best_prob` — how decisive the HMM prediction is
Conviction Score	0–100 composite score gating trade entry and determining leverage
Leverage Band	Leverage level (0x/10x/15x/25x/35x) mapped from conviction score
T1 / T2 / T3	Partial take-profit targets — 25% / 50% / remainder of position
Trailing SL	Stop-loss that moves with price peak, never backward
ATR	Average True Range — measure of volatility (price range per bar)
Kill Switch	Portfolio-level circuit breaker (10% drawdown in 24h → all positions closed)
Coin Tier	Classification A (trade) / B (monitor) / C (avoid) by walk-forward Sharpe
VWAP	Volume-Weighted Average Price — intraday fair value
Funding Rate	Periodic payment between long/short holders in perpetual futures
Open Interest (OI)	Total outstanding contracts — rising OI confirms directional conviction
Order Flow	Taker buy/sell imbalance — who is aggressing the book
Cumulative Delta	Running sum of (buy volume - sell volume)
BotSession	One complete bot run (start → stop) with aggregated performance metrics
ROI	`totalPnl / totalCapital × 100` — return on capital deployed
IC	Information Coefficient — correlation between signal and forward return
Walk-Forward	Out-of-sample backtesting: train on past, test on future, roll window
Paper Trade	Simulated trade on testnet — no real money, used by SaaS users

Live Trade	Real futures trade on CoinDCX with actual capital
Isolated Margin	Position margin isolated from rest of account — limits loss to trade capital
AES-256-GCM	Military-grade authenticated encryption used for API key storage
Prisma	TypeScript ORM (Object-Relational Mapper) for PostgreSQL
NextAuth	Authentication library for Next.js — handles JWT sessions
Engine API	Flask REST server bridging Python bot ↔ Next.js dashboard
syncEngineTrades	Function that syncs tradebook.json → PostgreSQL Trade table
ENCRYPTION_KEY	64-char hex env var used as AES-256-GCM key for API key encryption

---

Document generated: March 2026 Project: Regime-Master (HMMBOT) | GitHub: https://github.com/nikhildha/Synapticbots